Noto Peninsula: An Interactive 3D Landscape

Project Report

Group 12
LEE Hyunjin, TAN Jiming

HKUST - COMP5411 - Fall 2026
12 November 2026

Background

The project combines a civil-engineering interest in real terrain with JavaScript and GLSL programming. Both members began without prior computer-graphics experience. The objective is a navigable regional landscape whose geometry, materials and environmental effects can be examined interactively in a browser.

Project description

The viewer covers approximately 33.23 by 23.14 km of the Noto Peninsula, Japan, within 37.33-37.54 degrees N and 137.00-137.38 degrees E. A GSI 1 m elevation dataset supplies the terrain. Geological reference data guide rendering categories, while a 2024 vegetation classification guides representative plant placement. The application uses Three.js/WebGL2 and custom shaders; it does not use a full game engine.

Users rotate, pan and zoom, choose a finest terrain spacing, adjust vertical exaggeration, set date/time and weather, and inspect night scenes with a pointer flashlight. The implementation integrates adaptive terrain patches, geological materials, instanced vegetation, animated water and surf, sunlight, rain, elevated moving clouds and optional temporal reconstruction. A 5 m finest-spacing limit and automatic rendering/vegetation budgets are the defaults.

This report describes the implementation as of 12 November 2026. It emphasizes geometry continuity, coherent coastal animation and measured rendering costs, addressing the progress-review priorities. The scene is a geographical visualization: geological blends, plant representation, clouds and ocean motion are illustrative, not engineering or meteorological predictions.

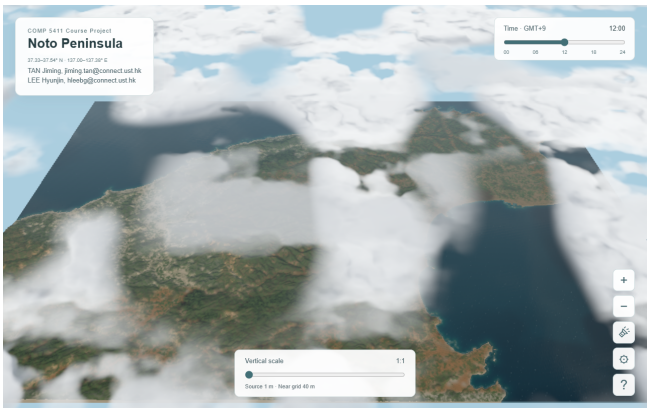


Figure 1. Regional terrain and elevated cloud layers.

Existing technology and system design

Three.js supplies scene objects, WebGL resource management, cameras, instancing and shadow-map infrastructure. Project modules construct terrain topology and implement geological shading, ocean integration, atmosphere/cloud shaders, scheduling and reconstruction. The ocean inverse-transform organization is adapted from David Li's MIT-licensed WebGL waves code; reflected-camera mathematics follows the attributed Three.js examples. These dependencies are distinct from the project's AI-assisted integration code.

Geometry processing: local, asynchronous level of detail

Rendering the whole crop as a 1 m grid would require approximately 1.54 billion surface triangles before walls or coastal refinement. Source-file tiles therefore supply samples to a separate hierarchy of display patches, with at most 128 base-grid cells per side before coastal subdivision. Spacing doubles from the user limit, for example 5/10/20/40 m. This keeps nodes nested even when the finest spacing is 3, 5, 6 or 7 m. Projected spacing, distance and resource budgets determine which patches split; separate split/merge thresholds and a view buffer reduce oscillation during small camera turns.

A dedicated Worker resamples heights, constructs land/water meshes, clips coastal triangles, stitches mixed-detail boundaries and prepares boundary normals. Geometry arrays return as transferable buffers. The main thread keeps the complete old coverage until a bounded local replacement and its affected seams are ready. Ordinary camera motion does not discard useful completed work; a changed user grid or obsolete topology does. This separates input responsiveness from expensive topology construction without moving the WebGL renderer into the Worker.

```
// Condensed control flow; actual modules retain ownership and error checks.
transaction = chooseLocalReplacement(livePatches, desiredPatches);
prepared = await worker.buildGeometryAndSeams(transaction);
if (stillApplicable(transaction, livePatches, userGrid)) {
  replaceCoverageAtomically(transaction, prepared);
}
```

Coastline and mixed-resolution continuity

Both land and water clip against the same zero contour of an 8 m sampled/smoothed signed-distance field. A narrow coastal band receives extra subdivision even when inland patches are coarse. Fine boundary vertices match the actual coarse surface, and animated water boundaries interpolate the same coarse endpoint displacements. Collision queries and plant roots use the displayed triangles. A visual 64 m shore-contact profile joins land to mean sea level, with the nearest 16 m held at sea level; this is not surveyed bathymetry.

Materials, plants and compact buffers

Triplanar PBR sampling reduces stretching on slopes. A categorical neighborhood blends geological appearances without treating category identifiers as continuous numbers. Deterministic texture rotations/offsets, mipmaps, explicit gradients and footprint-based shader detail reduce repetition and minification artifacts. Geological resolution is roughly 150 m, distinct from the 1 m DEM. Trees use deterministic locations, shape variants and distance-dependent mesh/card representations. Nearby grass uses instanced blades; distant cover remains in the material shader.

Eligible meshes store indices as Uint16Array; references of 65535 or greater retain Uint32Array because WebGL2 reserves the 16-bit value 65535 for primitive restart. This preserves triangle order, positions and normals, including after seam rebuilding. It reduces index storage and transfer size, not the number of triangles or draw calls.

Design and implementation: animated water

A hybrid geometry and spectral shading pipeline

Three long directional swells displace the sea mesh. A finer coastal strip adds shoreward-moving breaker crests while the exact contact contour remains stationary. Two independently seeded 128 by 128 wind-wave spectra, with 1024 m and 173 m domains, evolve on the GPU. Conjugate Fourier coefficients produce real fields, and horizontal/vertical inverse Stockham passes reconstruct displacement-related fields. Choppy derivatives supply slopes and crest compression for shading.

```
omega(k) = sqrt(9.81 * length(k))
h(k,t) = h0(k)*exp(-i*omega*t) + conj(h0(-k))*exp(i*omega*t)
field = inverseFFT(h)
// Spectral slopes/crest shading complement analytic mesh displacement.
```

The FFT fields are shading inputs, not a fully spectral displaced mesh. They update at most 20 Hz under the shared animation clock; unchanged time or invisible water skips the update. Analytic motion remains available if floating-point targets are unsupported. Independent spatial scales and continuously blended phase charts reduce obvious periodic repetition. Rotated slopes are transformed back to model coordinates; footprint and slope-variance filtering suppress unresolved fine detail.

Water color combines angle-dependent Fresnel reflection, absorption, sun highlights and foam. A mirrored camera renders a coarse terrain proxy into a 512 by 288 mipmapped target, capped at 150,000 proxy triangles. Captures respond to camera/lighting changes, are normally throttled to 10 Hz, with a final settling capture, and stop once stable. Sky and cloud optical columns provide reflected environment color. Detailed plants and geological surface shading are deliberately absent from the reflection proxy.

Representative contribution: coherent, irregular surf

A simple shoreline-distance band can resemble a painted line: geometric crests, white foam and spray may also appear unrelated. The current implementation uses the same shoreward phase for geometry and foam and adds a deterministic travelling wave-group envelope. In model metres and animation seconds, the envelope is:

```
q = 0.5 + 0.28*sin(0.013*x + 0.009*z - 0.13*t)
        + 0.22*sin(0.023*x - 0.017*z + 0.21*t)
E = 0.25 + 0.75*q*q                  // E is bounded by [0.25, 1]
breakerHeight = E * crestShape * shoreBand * weatherAmplitude
```

This makes groups of breaking waves strengthen and weaken along the coast. Geometry, foam and wet wash use the same strength variation. Analytic normals include the envelope gradient, and CPU height queries use the same expression. Spray evaluates the envelope at its emission time using t minus particle age, so particles do not vanish merely because a wave set weakens during flight. The existing upper displacement bound and stationary shoreline are retained; the change adds no texture, render target or independent timer.

In the isolated eight-crest regression probe, consecutive crest heights range from 0.362 to 0.982 m. Maximum sampled CPU/GPU displacement disagreement is 1.92e-6 m and the normal-component difference is 2.08e-4; all sampled shoreline-contact offsets are zero. This validates the sampled implementation consistency, not physical surf accuracy. The model does not solve shallow-water flow, overturning surfaces, fluid-solid collision or sediment transport.

Sunlight, weather and temporal reconstruction

Lighting and elevated animated clouds

A NOAA-style fractional-year approximation computes solar azimuth/elevation from the selected date and GMT+9 time at the region center. Clear, cloudy and rainy presets change lighting, fog, water and rain coherently. A shared analytic atmosphere supplies visible sky and water reflection, with directional sunrise/sunset grading. Local shadow-map coverage follows the view and reuses cached maps where possible; this is an approximation rather than a complete atmospheric-scattering solution.

Cloud density occupies two physical altitude bands, 1800-3200 m and 5000-5700 m, independent of terrain exaggeration. A generated 128-cubed density field is advected by different wind vectors. Camera rays integrate density in front-to-back order and stop at scene depth, allowing views below, inside and above the layers. Reduced-resolution integration and depth-aware composition limit cost. Two optical-column maps approximate cloud appearance in water reflections. Clouds do not cast terrain shadow maps.

Rain that stays in the world

A camera-local rain box is inexpensive, but translating every drop by the camera position makes rain slide with the viewer. The revised shader retains deterministic world-space phases and uses a periodic volume around the camera to choose nearby copies. For seed p0, velocity v, camera anchor a and volume extent L, the head position is mod(p0 + v*t - a + L/2, L) - L/2 + a: the anchor cancels between wraps. Per-drop fall speeds of 9-15 m/s and exposure lengths of 0.035-0.055 s vary streaks; boundary and near-camera fades hide recycling. The GPU animates a fixed population without per-frame particle-buffer updates.

Weather preset changes remain immediate. Rain is an appearance effect: it does not collide with the DEM or produce simulated puddles, runoff or individual water-impact ripples. Sunlight remains approximate and is not a photometric validation of the real site.

Rejecting stale history when surfaces become visible

Optional temporal reconstruction shades the scene at 75% of output width and height, reprojects static surfaces into a full-output history and resolves the result. Current depth reconstructs world position; the previous view-projection matrix locates history. Depth-derived normals and linear depth test whether history belongs to the same surface. All four history taps used by bilinear color filtering must agree, preventing a thin foreground object from contaminating the newly revealed background.

```
valid = insidePreviousView && allFourTapsAgree(depth, normal);
history = clamp(historyColor, currentNeighborhoodMin,
                              currentNeighborhoodMax);
resolved = mix(currentColor, history, valid ? historyWeight : 0);
```

A 3 by 3 current-color neighborhood clamps accepted history; luminance disagreement reduces its weight. Significant lighting, geometry, visibility and scale changes reset history. Water, foliage, particles and animated coast materials use the current frame only; moving clouds are composed after static history. The method has no per-object motion vectors and does not claim universal ghost-free reconstruction.

Validation and idle behavior

The scheduler separates full scene updates from animation-only frames. Default animation stops 30 seconds after interaction; finite reconstruction settling does not advance sleeping effects. Worker construction and cooperative plant placement keep expensive preparation separate from the input/render path.

Geometry fixtures cover coverage, mixed-LOD seams, displayed-height queries, normals and index/Worker boundaries. Current browser checks pass ocean displacement/normal parity, grouped crests, rain world trajectories and sleep. Existing reconstruction fixtures exercise thin-occluder disocclusion and lighting changes. These checks do not replace long-session or multi-device acceptance.

Visual results

These captures are from the running application. The coastal view shows the water/land relationship; the low-sun view shows shared atmosphere, clouds and reflection. Still images illustrate appearance, while motion and correctness are checked separately in the browser fixtures.

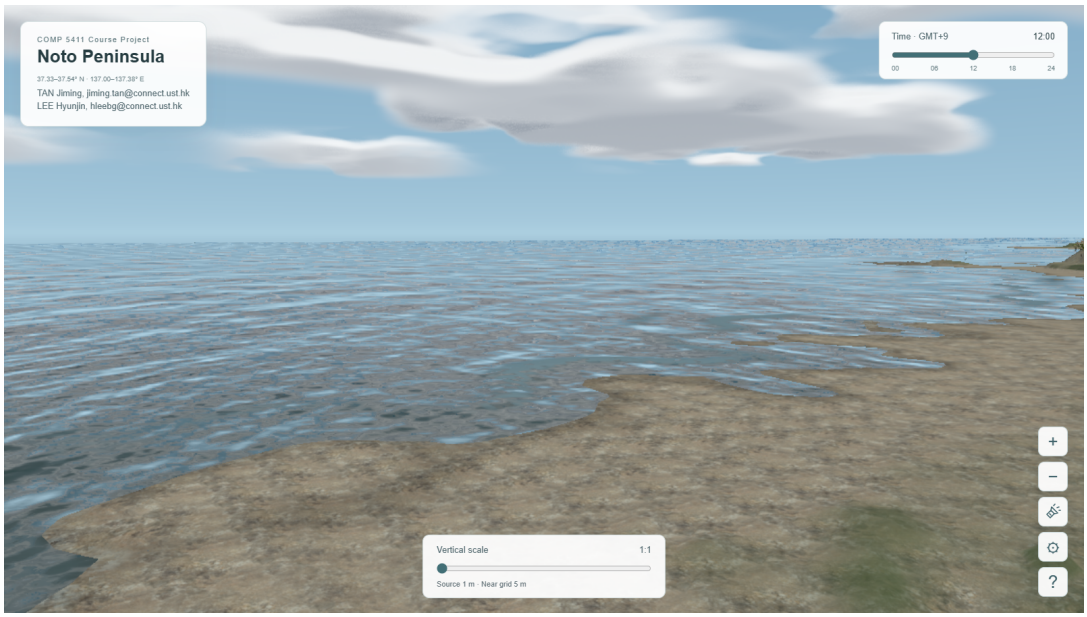


Figure 2. Current coastal water and surf. Long swells, spectral slopes, reflected surroundings and the shared breaking-wave envelope contribute different scales of appearance.



Figure 3. Time-dependent atmosphere, clouds and reflected light. The same time and lighting state drive the environment and water.

Results and performance evaluation

The progress review identified both water appearance and apparent low frame rate. A recording can skip frames independently of browser interaction, but it cannot explain away rendering costs. The following controlled browser measurements separate GPU pass time from CPU submission and asynchronous LOD work. They are not a measurement of the submitted Zoom recording or a guarantee of interactive frame rate on other hardware.

A controlled evaluation used Microsoft Edge 157.0.4341.46 and the browser-reported Intel Graphics device through ANGLE/D3D11. Output was 1280 x 800 pixels, with 960 x 600 temporal input, a 5 m finest-detail limit, 1:1 elevation scale and high vegetation detail. Lighting was clear noon; rain was validated separately. Automatic quality feedback and tree-wind phase were frozen; water/cloud clocks advanced deterministically. Each settled view, with cached shadows/reflections and no continuous lighting changes, used two repetitions of 16 warm-up and 32 sampled frames; no screen recorder was involved.

View	GPU total before / after (ms)	Main scene after (ms)	FFT after (ms)	CPU submit after (ms)
Soil slope	29.01 / 26.20	24.56	0.16	3.90
Coast	7.22 / 7.51	6.18	0.16	1.62
Regional	29.59 / 29.28	27.77	0.16	2.69

GPU values are means of asynchronous WebGL timer-query measurements summed over render passes. The FFT value is amortized over every sampled frame, including frames between its 20 Hz updates; water shading remains part of the main scene. CPU values measure render submission and exclude asynchronous LOD construction. These settled measurements are not end-to-end interaction latency or video frame rate. The short single-device before/after runs establish cost and variability, not a statistically demonstrated speedup. Before is ef9b960; after includes grouped surf and revised rain.

View	Patches	Grid range (m)	Triangles drawn	Draw calls
Soil slope	130	5-160	7.13 M	209
Coast	100	5-160	1.47 M	100
Regional	35	40	5.81 M	134

Terrain detail stayed spatially adaptive: the regional view used coarser patches, while nearby terrain reached the permitted fine spacing. The full pose-to-settled sequence took 18.7-28.9 s in this ordered route, including tile access, Worker construction, cache changes and stability waits. This is the time until all requested refinements finish, not time with a blocked interface; coarser coverage remains during progressive commits. Slow completion on large jumps remains a limitation.

The dominant GPU cost was main-scene rendering rather than the spectral update alone. Therefore, reducing FFT size is unlikely to address the largest cost in these views. The measured rendering workload can contribute to perceived sluggishness independently of recording; this experiment does not isolate how much Zoom affected the earlier video. The final integration completed without page/shader errors, and the frame counter and animation clocks stopped during the idle check.

A separate controlled check on 4 November compared index compaction with identical topology at three fixed views: active terrain index bytes fell by 41.91% on a soil view, 37.43% at the coast and 32.51% in a regional view. The images were pixel-identical in that check. This is an index-storage result, not total VRAM saving or an FPS improvement; positions, triangle counts and draw calls were unchanged.

Key challenges and achieved scope

Representative contribution: continuity during refinement

The difficult part of adaptive terrain is not choosing a smaller spacing; it is changing topology while keeping coverage, coast contact and navigation consistent. Source tiles and display patches have different boundaries. Replacing an entire visible set at once delays nearby improvement, while displaying partially prepared children exposes holes. The current design separates selection from construction, retains complete parent coverage, and commits local parent/descendant groups together with affected seams. Fine edges conform to the displayed coarse surface rather than merely sharing source samples.

This creates three useful invariants: every visible location has coverage while loading; neighboring land and moving-water boundaries agree; and collision/plant roots sample the geometry actually displayed. Camera motion can reprioritize work without repeatedly restarting it. A user resolution change invalidates incompatible transactions. Resource and construction budgets still limit how quickly detail becomes available; refinement is asynchronous rather than instantaneous.

Proposal milestones

Stage	Planned focus	Current implementation
Terrain / Week 4	Geometry and materials	DEM display patches, geological PBR, blended boundaries, anti-repetition sampling and filtering.
Week 5	Vegetation	Classification-guided instances, tree variants, grass, root alignment and independent visibility controls.
Week 6	Animated water	Long-swell geometry, GPU spectral shading, cached reflection, grouped surf, wetness and spray.
Weeks 7-8	Sun and weather	Date/time lighting, local shadows, elevated moving clouds, GPU rain and night flashlight.
Week 9	Adaptive detail	Worker construction, local LOD transactions, caches, shader detail, compact indices and sleep.
Week 10	Integration and evaluation	Targeted regression tests, representative performance measurements and this report; final presentation and broader acceptance remain separate deliverables.

Limitations

The 1 m source grid does not imply 1 m geological, vegetation or coastline accuracy. One unavailable fine input is handled through coarser available data; complete fine-source coverage is not claimed. The model base at -100 m is a truncation surface, not measured seabed. The approximate shore profile is a display treatment. Reflective detail, cloud integration and plant representation are deliberately bounded. Temporal reconstruction cannot reconstruct dynamic details from invalid history.

Evaluation covers representative views on one exposed browser/GPU configuration. Extended routes, long sessions, additional GPUs and observer studies remain unverified. The application requests high-performance graphics, but the browser and operating system choose the actual adapter; JavaScript cannot force a device number. Alternative reconstructions such as voxels, NeRF or Gaussian splatting, seasonal snow and a separate walking mode are outside the implemented scope.

Code structure and reproducibility

Paths below are relative to the project root. Open index-1m.html directly, or use start-viewer.cmd / python scripts/serve.py for loopback HTTP. Runtime assets use relative paths and are distributed separately from small source files where needed; see README.md and assets-manifest.json. The report source and its figures are included with the project.

Relative path	Responsibility
index-1m.html; src/styles.css	Viewer controls and layout.
src/terrain.js	Scene integration, navigation, cache budgets and shared frame scheduler.
src/terrain-lod.js; src/terrain-refinement.js; src/geometry-worker.js	Patch selection, local replacement transactions and background geometry work.
src/tile-geometry.js; src/coastal-geometry.js; src/model-volume.js	Mesh generation, coastline clipping, seam conformity, closed volume and index storage.
src/rock-materials.js; src/vegetation.js; src/ground-grass.js	Geological shading and representative vegetation.
src/ocean-spectrum.js; src/water-motion.js; src/scene-effects.js	GPU spectra, geometric/optical water, surf and spray.
src/ocean-reflection.js; src/ocean-backdrop.js	Bounded terrain reflection and illustrative offshore horizon.
src/environment.js; src/sky-atmosphere.js; src/cloud-layers.js	Sunlight, rain, atmosphere and elevated clouds.
src/temporal-reconstruction.js; tests/	Static history validation and focused CPU/browser regression fixtures.
data/; assets/material-library/; assets/vegetation/; vendor/	Generated runtime data, local assets and attributed third-party libraries; consult the asset manifest for complete paths.
docs/; scripts/build_project_report.py	Reports, figures, structured report source and PDF builder.

AI assistance and external contributions

Human contributions documented in the development record include the regional/geological visualization objective, selection of source inputs and requirements, prioritization of water/interaction/cloud work, repeated inspection of the demo, identification of visual/performance defects and decisions on acceptable behavior. These are project design and evaluation contributions; they should not be presented as independently handwritten shader code.

Codex assisted code generation, revision, debugging and test construction across the custom terrain/Worker/LOD pipeline, material and vegetation integration, water/surf/FFT integration, atmosphere/clouds, environment/rain, temporal reconstruction and scheduling. The wave-group and rain changes described here, performance harnesses and this report were also prepared with Codex assistance under human direction. The report does not assign unverified independently handwritten modules to either member.

Three.js is an external MIT-licensed runtime, not student-authored. The inverse Stockham transform organization in ocean-spectrum.js derives from the pinned dli/waves implementation, while spectrum initialization, viewer integration and budgets are project-specific AI-assisted work. Reflection mathematics follows Three.js Water/Reflector references. Texture/model assets and geospatial inputs retain their own licenses; their use does not make the complete project CC0.

References and source records

The following are implementation and data sources, not claims of new rendering algorithms. Detailed asset authors, versions and license texts are retained in LICENSES-AND-SOURCES.md, TEXTURE-SOURCES.md, THREE-LICENSE.txt and the source manifests.

[1] Three.js. WebGL scene/runtime, Water.js and Reflector.js reference implementations, MIT license. github.com/mrdoob/three.js. Attributed reference version: r186.

[2] David Li. WebGL waves, simulation.js, MIT license. Pinned commit 1313f464c2df7d1c69852577248b58fa29074c38. github.com/dli/waves. The project reuses the inverse-transform organization, not the complete ocean application.

[3] Geospatial Information Authority of Japan. Fundamental Geospatial Data DEM service. GSI DEM service. Original terrain inputs and metadata are retained with the project assets.

[4] Geological Survey of Japan; NIED K-NET/KiK-net; MLIT Noto Reconstruction Office; J-SHIS. Geological maps and borehole/reference inputs. Exact source records accompany noto_terrain_materials_package/. Rendering categories are project-curated summaries.

[5] University of Tsukuba. 2024JPN_v26.03 vegetation classification. Publisher data page. EOxCloudless imagery contains modified Copernicus Sentinel data 2024, credited to EOX IT Services GmbH under CC BY-NC-SA 4.0; it supports vegetation interpretation, not a live streamed imagery layer.

[6] Poly Haven and ambientCG. Material/model assets and leaf atlas under their documented CC0 terms. Per-asset authors and download records accompany the files.

[7] COMP5411 course materials, Fall 2026. Project proposal, project guidelines, frame operations and advanced pipeline stages. The implementation applies geometry processing, GPU shading, instancing, filtering, depth testing, frame-buffer operations and level-of-detail concepts using the supported WebGL2 pipeline.

Conclusion

The project realizes the proposal as an integrated browser landscape rather than a static terrain mesh. Its main technical work lies in keeping adaptive geometry continuous, combining multiple scales of water appearance within a bounded rendering pipeline, and maintaining coherent interaction, lighting and animation. Progress feedback led to stronger water/sky integration, coherent wave groups, world-anchored rain and explicit performance reporting. The remaining limitations are stated separately from the demonstrated features and targeted measurements.